

CI/CD Pipeline

Three terms that get used for each other, one pipeline that connects them, and the build file that makes any of it possible in a Java project.

1. What is Continuous Integration (CI)?

Continuous Integration (CI) is the practice of frequently merging code changes into a shared codebase and automatically building and testing them to find problems early.

It avoids the situation where developers work separately for a long time and discover conflicts, broken tests, or unexpected issues only when they finally combine their changes.

Without CI → merge once every few weeks, and spend a day on it
With CI → merge several times a day, and find out in ten minutes

Integrating often is the practice. The server is only a tool. A team with Jenkins running nightly on a branch nobody merges into is not doing Continuous Integration; a team that merges to `main` four times a day and runs the tests each time is, whatever the tool.

Four things have to be true:

Requirement	Why
One shared branch	Long-lived branches are the thing being avoided
Every push triggers a build	An untested commit is an unknown
Tests run automatically	A suite nobody runs proves nothing
A broken build is fixed first	Everything after it is built on an unknown

The measure is the time from pushing to knowing. Under ten minutes and people wait for it; over thirty and they stop looking, which removes the point.

2. What is Continuous Delivery?

Continuous Delivery (CD) is the practice of keeping every code change ready to be deployed to production at any time. The pipeline builds the application, creates a deployable artifact, deploys it to a test environment, and runs tests to make sure it is ready for production.

Production is still a button. Somebody chooses when to press it, and that choice is about business timing, not about whether the software is ready.

What it needs beyond CI:

Requirement	Detail
An automated deployment	The same script runs for every environment
Environments that match	Staging that differs from production proves less
Externalized configuration	One artifact runs everywhere, configured at start
Migrations handled	Schema changes apply as part of the deploy

The test is how long a release takes. If deciding to release means a plan, a change window and a checklist, it is not Continuous Delivery, however automated the build is.

3. What is Continuous Deployment?

Continuous Deployment removes the last manual step: every change that passes the pipeline goes to production automatically, with nobody approving it.

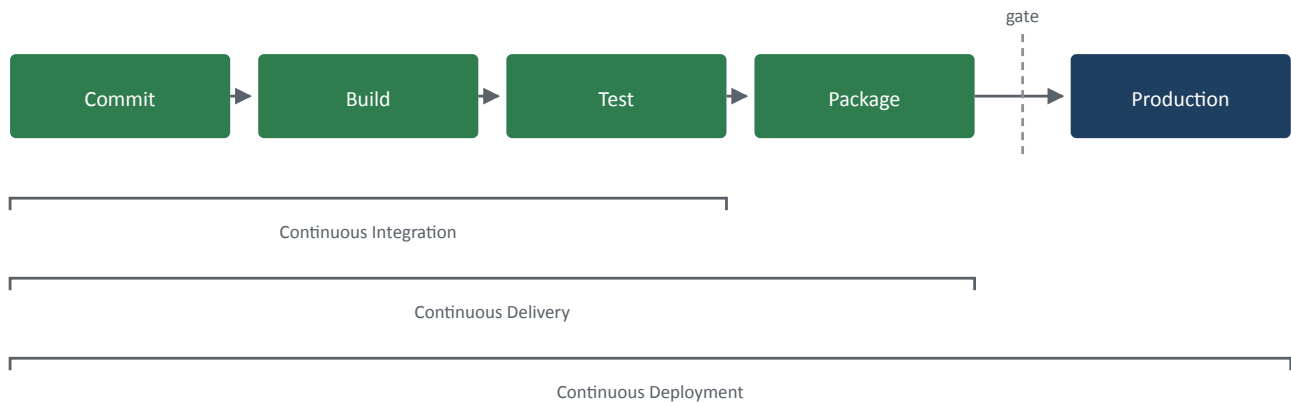
It is one step past Continuous Delivery, and the step is a small change in tooling and a large change in confidence.

Requirement	Why it is not optional
Strong automated tests	Nothing else stands between a commit and users
Monitoring and alerting	Problems are found in production or not at all
Feature flags	Code can ship while the feature stays off
Fast, tested rollback	The first response to a bad release, every time
Progressive rollout	Canary or blue-green, so a bad release reaches few users

Feature flags are what make it workable. They separate deploying code from releasing a feature, so an unfinished feature can be merged and deployed with no risk, which in turn is what allows merging often.

Most teams stop at Continuous Delivery, on purpose. Regulated work, a customer who schedules upgrades, or a release that has to match an announcement all make the button worth keeping. Saying that in an interview is better than treating Continuous Deployment as the goal for everybody.

4. CI vs Continuous Delivery vs Continuous Deployment



The three names describe how far the automation reaches. The dashed line is the human decision, and Continuous Deployment is the same pipeline with that line removed.

	Continuous Integration	Continuous Delivery	Continuous Deployment
Ends with	A verified build	A deployable artifact	Running in production
Production deploy	Not covered	Manual, one button	Automatic
Answers	Does it work?	Could we ship it?	Have we shipped it?
Needs	Tests and a shared branch	Plus automated deploys	Plus flags and monitoring
Risk per release	—	Held until someone decides	Small, because releases are small

The short answer for an interview: CI is about merging and proving; Continuous Delivery is about always being ready to release; Continuous Deployment is about releasing without asking. The abbreviation CD is ambiguous and it is worth saying which one is meant.

5. What is a CI/CD pipeline?

A CI/CD pipeline is an automated process that takes code from a commit, builds and tests it, and deploys it to the required environment. If any stage fails, the pipeline stops.

Three important things make it a proper pipeline:

- **It fails fast:** Cheap and common checks run first. For example, a compile error should be caught before running a 10-minute integration test.
- **It builds once:** The same artifact that was tested is deployed to each environment. This avoids testing one version and deploying a different version.
- **It is triggered automatically:** A code push, pull request, tag, or schedule can start the pipeline, so developers don't have to run it manually.

6. Why should pipeline configuration live in Git?

Because the pipeline is part of the software. Everything that is true of the code should be true of the way the code is built.

Tool	File
GitHub Actions	<code>.github/workflows/*.yaml</code>
GitLab CI	<code>.gitlab-ci.yml</code>
Jenkins	<code>Jenkinsfile</code>
CircleCI	<code>.circleci/config.yml</code>

Five things follow from the file being in the repository:

Property	What it gives
Reviewed	A build change goes through the same pull request as the code
Versioned	<code>git log</code> says who changed the build, when, and why
Branch-aware	A branch can change its own pipeline and prove it before merging
Reproducible	An old commit checks out the pipeline that built it
Recoverable	Nothing is lost with the server it ran on

The alternative is a form in a web interface, and its problems are all the same problem: the configuration is state nobody has a copy of. A change cannot be reviewed, the history is whatever the tool chose to keep, and testing a change means changing it for everybody at once.

A branch can change its own build. This is the property people notice first. Adding a stage is a normal pull request: the branch runs with the new stage, the result is visible in the review, and merging makes it the rule for everyone.

Secrets are the exception. The pipeline definition belongs in Git; the credentials it uses do not. They live in the CI system's secret store or a vault, and the file refers to them by name only.

```
- run: ./deploy.sh
  env:
    REGISTRY_TOKEN: ${ secrets.REGISTRY_TOKEN }
```

Anything committed to a repository is in its history forever, so a secret pushed once has to be rotated, not deleted.

7. Pipeline stages

The names vary and the order rarely does. What follows is the full set. A given project runs a subset of it, and the stages most often missing are the security scan and the acceptance tests.

Stage	What it does	Fails when
Checkout	Fetches the commit	Rarely
Build	Compiles the code	A compile error
Unit test	Runs the fast tests	Logic is wrong
Static analysis	Lint, style, complexity, coverage	Rules are broken
Package	Produces the jar or image	Packaging is misconfigured
Integration test	Runs against real dependencies	Wiring is wrong
Security scan	Checks dependencies and images	A known vulnerability
Publish	Pushes the artifact to a registry	Credentials or permissions
Deploy to staging	Installs it somewhere real	Environment or config
Acceptance test	Proves it works deployed	Behavior is wrong
Deploy to production	Releases it	The last gate

The ordering principle is cost. Put the stage that is quickest to run and most likely to fail as early as it can go, so that feedback arrives in seconds rather than after a full pipeline.

Stages that do not depend on each other should run at the same time. Unit tests, static analysis and a dependency scan need nothing from one another, and running them in parallel often halves the pipeline.

```
jobs:
  verify:
    steps:
      - run: ./mvnw -B verify
  analyze:
    steps:
      - run: ./mvnw -B verify sonar:sonar
```

The image-specific stages, building and tagging and scanning a container, are covered in Docker in CI/CD.

8. Build automation

Build automation is turning source into a tested, packaged artifact with one command, the same way on every machine.

```
./mvnw -B verify
```

That one command resolves dependencies, compiles, runs the tests and produces the jar. The developer runs it, and the pipeline runs exactly the same thing, which is what stops the phrase "it builds on my machine".

What it removes	How
Manual steps in an order	The lifecycle defines the order
Different results per machine	Declared dependencies and a pinned tool version
Forgotten tests	Testing is a phase of the build, not a separate task
A build nobody else can run	The whole definition is in the repository

Use the wrapper. `mvnw` and `gradlew` are checked in and download the exact build tool version the project expects, so the pipeline does not depend on what happens to be installed on the runner.

-B matters in CI. Batch mode turns off interactive prompts and the progress animation, which otherwise fills the log with control characters.

9. Why do we use pom.xml in Java?

`pom.xml` is the Project Object Model: the file that tells Maven what the project is, what it depends on, and how to build it.

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>net.rohitshukla</groupId>
  <artifactId>payroll</artifactId>
  <version>1.4.2</version>
  <packaging>jar</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
  </parent>
</project>
```

```

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>

```

It answers four questions that a build otherwise answers by hand:

What is this project? `groupId`, `artifactId` and `version` name it uniquely, which is also how anything else depends on it.

What does it need? Dependencies are declared, not downloaded and committed. Maven fetches them and their dependencies, so declaring `spring-boot-starter-web` brings Tomcat, Jackson and the rest without naming them.

How is it built? The lifecycle is fixed, so every Maven project is built the same way:

Phase	Does
<code>validate</code>	Checks the project is correct
<code>compile</code>	Compiles the source
<code>test</code>	Runs unit tests
<code>package</code>	Produces the jar
<code>verify</code>	Runs integration tests and checks
<code>install</code>	Copies it to the local repository
<code>deploy</code>	Publishes it to a remote repository

Each phase runs every phase before it, which is why `verify` alone is enough in a pipeline.

Which version of everything? The parent, or `dependencyManagement`, sets versions in one place. That is why the dependency above needs no version: the Spring Boot parent already picked one that works with the rest.

Dependencies also carry a scope, which says when each one is needed. Getting it wrong is how a test library ends up shipped inside the jar:

Scope	Available at
compile	Everywhere. The default
runtime	Running, not compiling. A JDBC driver
provided	Compiling, supplied by the container at run time
test	Tests only. JUnit, Mockito

Why it matters for CI, which is the reason it is in this document: the pipeline needs no build knowledge at all. It checks out the repository and runs one command, and the file says everything else. A build described in a CI configuration instead of in `pom.xml` cannot be run on a laptop, which is where it needs to be run first.

10. Pipeline caching

A CI runner is a fresh machine. Without caching it downloads every dependency on every run, and for a Spring Boot project that is several hundred jars and several minutes, repeated on every commit all day.

The fix is to keep the local dependency directory between runs.

```
- uses: actions/cache@v4
  with:
    path: ~/.m2/repository
    key: maven-${{ hashFiles('**/pom.xml') }}
    restore-keys: maven-
```

The key is the whole design. It is a hash of the files that decide what the dependencies are, so the cache is reused exactly while `pom.xml` is unchanged and invalidated the moment it changes.

restore-keys is what makes it useful in practice. An exact miss would otherwise mean downloading everything again. With a prefix fallback, a changed `pom.xml` restores the most recent cache anyway, and only the new dependencies are fetched.

Tool	Cache this	Key on
Maven	<code>~/.m2/repository</code>	<code>**/pom.xml</code>
Gradle	<code>~/.gradle/caches</code> , <code>~/.gradle/wrapper</code>	<code>**/*.gradle*</code> , <code>gradle-wrapper.properties</code>
npm	<code>~/.npm</code>	<code>package-lock.json</code>

On GitHub Actions the Java action does all of it in one line, which is the version to reach for first:

```
- uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: 'temurin'
    cache: maven
```

Three rules keep a cache from causing the problems it was meant to solve:

Cache inputs, never outputs. Dependencies are downloaded and identical everywhere. The `target/` directory is a build result, and restoring it means a later build reuses classes compiled from code that is no longer there.

The build must work with an empty cache. A cache is an optimization. If a cold run fails, the pipeline was depending on leftover state and will fail the first time the cache is evicted.

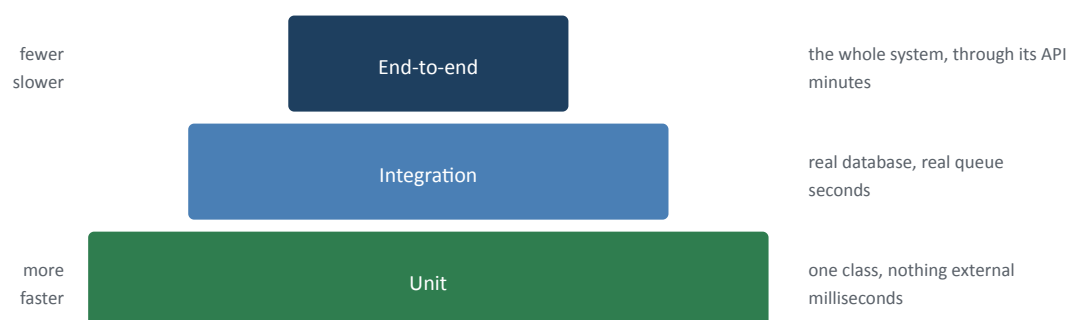
Key on the manifest, not the branch. A key that never changes serves stale dependencies after an upgrade, and the resulting failure looks like a code problem rather than a cache problem.

Caching a container image's layers is a different mechanism with different flags, and it is covered in Docker in CI/CD.

11. How does automated testing work in CI?

Automated testing in CI means the pipeline runs the suite on every commit and refuses to go further when it fails. Nobody chooses to run the tests, and nobody can choose to skip them.

The tests are not one thing. They come in layers, and the layers have very different costs.



The shape is the point. Most of the confidence should come from the layer that is cheapest to run and easiest to diagnose.

A common shape is roughly seventy percent unit, twenty percent integration and ten percent end-to-end. The exact numbers matter less than the direction: an inverted pyramid, where most of the confidence comes from slow end-to-end tests, is the failure mode the diagram warns about.

Layer	Covers	Runs	Fails when
Unit	One class, dependencies mocked	Every push, in the build	Logic is wrong
Integration	Several parts plus a real database or queue	After packaging	Wiring or SQL is wrong
End-to-end	The deployed system through its API	After a staging deploy	The whole flow is wrong

Run them in that order, and stop at the first failure. An end-to-end run against a build whose unit tests already fail is wasted time, and the failure it reports is harder to read than the one already sitting there.

In Maven the split is two plugins, and this is the Java-specific answer:

Plugin	Runs	Phase	Naming
Surefire	Unit tests	<code>test</code>	<code>PayrollServiceTest</code>
Failsafe	Integration tests	<code>integration-test</code> , reported by <code>verify</code>	<code>PayrollControllerIT</code>

That is why a pipeline runs `verify` and not `test`. `mvn test` stops after the unit tests; only `verify` reaches the integration tests and reports their failures.

Integration tests should use a real dependency, not a fake one. Testcontainers starts the actual database in a container for the test and throws it away afterwards, which removes the difference between what the test proved and what production does.

```
@SpringBootTest
@Testcontainers
class PayrollRepositoryIT {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");

    @Autowired
    PayrollRepository repository;

    @Test
    void savesAndReadsBack() {
        repository.save(new Employee("Rohit", 250));
        assertThat(repository.findAll()).hasSize(1);
    }
}
```

A flaky test is worse than a missing one. A suite that fails at random teaches everybody to press retry, and once that habit exists a real failure is retried too. Quarantine the flaky test, open a ticket, and fix the cause; do not add an automatic retry and call it solved.

Green does not mean correct. It means nothing that was checked is broken. The useful question after a bug reaches production is which layer should have caught it, and then writing that test before the fix.

12. How does static code analysis work in CI?

Static analysis reads the source without running it and reports problems the compiler accepts. In CI it runs as its own stage and can fail the build.

It is the second opinion that never gets tired, and it finds a different class of problem from the tests: the tests prove the code does what it should, and the analyzer says whether the code is something anyone can maintain.

What it reports uses the SonarQube vocabulary, which most tools now share:

Finding	Means	Example
Bug	Code that will behave wrongly	A possible <code>NullPointerException</code>
Vulnerability	A security flaw in our code	SQL built by string concatenation
Security hotspot	Security-sensitive, needs a human decision	A hardcoded path, a weak random
Code smell	Works, but is hard to maintain	A 300-line method, a duplicated block
Technical debt	The estimated time to fix all the smells	"4 days"
Coverage	How much of the code the tests execute	74% of lines
Duplication	Repeated blocks	6% duplicated lines

Technical debt is an estimate, not a measurement. Sonar prices each smell with a fixed remediation cost and adds them up. It is useful as a trend and meaningless as an absolute number, and treating it as a target produces changes made to move a number.

A quality gate is the pass or fail decision. It is a set of conditions checked after the analysis, and failing it fails the pipeline:

Condition	Typical value
Coverage on new code	At least 80%
Duplicated lines on new code	Below 3%
New bugs	Zero
New vulnerabilities	Zero
New security hotspots	All reviewed
Maintainability rating on new code	A

Gate on new code, not the whole codebase. A ten-year-old service will never reach 80% coverage overall, and a gate demanding it is switched off within a week. Requiring it only of the lines in this change is achievable on day one and improves the codebase steadily. Sonar names this mode "Clean as You Code", and it is the single most useful idea in the tool.

```
./mvnw -B verify sonar:sonar \
  -Dsonar.projectKey=payroll \
  -Dsonar.host.url=https://sonar.company.com
```

Coverage is measured during the test run, not by the analyzer. JaCoCo instruments the code while the tests execute and writes a report that Sonar then reads, which is why coverage is zero when the analysis runs without the tests.

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
  </executions>
</plugin>
```

80% coverage does not mean the code is tested. Coverage says a line ran, not that anything was asserted about it. A test with no assertions gives full coverage, and branch coverage is the more honest number because it asks whether both sides of every `if` were taken.

Tool	Finds
SonarQube	All of the above, with history and the quality gate
SpotBugs	Bug patterns in bytecode
PMD	Smells and questionable constructs in source
Checkstyle	Formatting and naming conventions
Error Prone	Bug patterns, at compile time
OWASP Dependency-Check, Snyk	Known vulnerabilities in dependencies

They are not alternatives. Checkstyle in the IDE, Error Prone in the compiler, SonarQube as the pipeline gate and a dependency scanner beside it is a normal arrangement, because each catches what the others do not.

13. Dependency management and security

Static analysis checks the code we wrote. Most of what ships is code we did not write, and that is the larger attack surface.

```
./mvnw dependency:tree
```

A Spring Boot service declares perhaps fifteen dependencies and ships several hundred, because each one brings its own. Those transitive dependencies are the ones nobody chose and nobody is watching.

Concern	The question
Known vulnerabilities	Does anything we ship have a published CVE?
Staleness	How far behind are we, and how hard is catching up?
Licensing	Is anything here incompatible with shipping a product?
Provenance	Did this come from the registry we think it did?

Scanning is a pipeline stage that fails the build.

```
- run: ./mvnw -B org.owasp:dependency-check-maven:check
```

Sonar and the tools beside it were covered in the previous section; for dependencies the common choices are OWASP Dependency-Check, Snyk and the scanning built into GitHub and GitLab. All of them compare the resolved dependency list against a vulnerability database.

Fix the version, not the symptom. When a transitive dependency is vulnerable, the answer is usually to force a newer one rather than to remove the library that pulled it in:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.2</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

`dependency:tree` is what finds the library that brought it in, which is the first thing to look at when a scanner reports something nobody recognizes.

Staying current is a security control, not housekeeping. Dependabot and Renovate open pull requests for updates, and because the pipeline tests each one the merge is cheap. A project that updates weekly absorbs a critical fix in a day; a project two years behind needs a migration first, during an incident.

An allowlist is necessary and dangerous. A vulnerability with no fix available cannot be acted on, so the build has to be able to continue. Every suppression should carry an expiry date and a reason, or the file becomes a list of things nobody will look at again.

14. Container and application security

Security is not one stage. Each layer is checked at the point where it is cheapest to fix.

Layer	Checked by	Stage
Our source	SAST, code review	Static analysis
Our dependencies	Dependency scanning	The previous section
The image	Image scanning	After publish
Secrets in the repository	Secret scanning	Every push
The running service	DAST, penetration testing	Against staging

Secret scanning belongs on every push, because a committed credential is in the history permanently and a scan that runs weekly finds it after it has been cloned. Tools such as gitleaks and the scanning built into GitHub run in seconds.

Four properties make a container image defensible, and all four are decided in the Dockerfile:

Property	How
A minimal base	Alpine or distroless. Less installed is less to patch
Pinned by digest	The scanned image and the deployed image are the same bytes
A non-root user	<code>USER app</code> , so a compromise does not start as root
No secrets in layers	Build secrets or run-time environment, never <code>ENV</code>

Image scanning itself, and where it sits in the pipeline, is covered in Docker in CI/CD; the point here is that it is one layer of five rather than the whole answer.

Run-time hardening is separate from the build. A read-only root filesystem, dropped capabilities and a security context are properties of how the container is run, and they belong in the deployment manifest.

A passing scan expires. The image does not change and the vulnerability database does, so anything long-lived needs a scheduled rescan of what is actually deployed, not only a scan at build time.

15. What is an artifact, and how does artifact management work in CI?

An artifact is the packaged output of a build: the single file the pipeline produces and the deployment consumes.

Artifact	Produced by
<code>payroll-1.4.2.jar</code>	<code>mvn package</code>
<code>payroll-1.4.2.war</code>	The same, with <code>war</code> packaging
A container image	<code>docker build</code>
An npm package, a zip, a Helm chart	Their own tools

Four properties define one, and all four matter to the pipeline:

Property	Meaning
Built once	From one commit, by one build
Immutable	Never edited after it is published
Identifiable	A version or digest names exactly these bytes
Stored outside	It outlives the pipeline run that made it

The last one is the point of artifact management. A jar sitting on a runner disappears when the job ends. Publishing it to a repository is what lets a deploy an hour later, or a rollback a week later, use the exact thing that was tested.

Repository	Holds
Nexus, Artifactory	Anything. Usually self-hosted
GitHub Packages, GitLab Registry	Artifacts beside the code
ECR, Artifact Registry, ACR	Container images
Maven Central	Public Java libraries

A Java project meets the same repository twice. It pulls dependencies from one and pushes its own output to one, and `mvn deploy` is the second half:

```
<distributionManagement>
  <repository>
    <id>releases</id>
    <url>https://nexus.company.com/repository/maven-releases/</url>
  </repository>
</distributionManagement>
```

Snapshot against release is the Maven question that gets asked.

	1.4.2-SNAPSHOT	1.4.2
Meaning	Work in progress toward 1.4.2	The released 1.4.2
Overwritten	Yes, on every build	Never
Resolved	Re-checked for a newer copy	Once, then cached forever
Deployed to production	No	Yes

A release must not depend on a snapshot. Doing so makes the release unreproducible, because the snapshot it was built against has since been replaced. Maven's release process refuses it for that reason.

The pipeline's use of all this is three rules:

Publish once, after the tests pass. Not before, and not again per environment.

Promote the same artifact. Staging and production install the identical version, so what was tested is what runs.

Keep what production has run. Retention policies that delete old artifacts also delete the thing a rollback needs. Expire snapshots and branch builds freely; keep releases.

16. Deployment approvals and gates

A gate is a condition that has to be satisfied before the pipeline goes further. There are two kinds, and the difference matters.

	Automated gate	Human approval
Decided by	A rule, evaluated by the pipeline	A person pressing a button
Examples	Quality gate, scan result, smoke test	A release manager, a change board
Cost per run	None	Minutes to days
Scales	Yes	No

```
jobs:
  deploy-production:
    environment: production      # protection rules apply here
    needs: [deploy-staging]
    steps:
      - run: ./deploy.sh --env=production
```

Naming an environment is what attaches the rules: required reviewers, a wait timer, and which branches may deploy to it. The pipeline definition stays in Git and the policy lives with the environment, so a change to either is visible.

Prefer an automated gate to a human one wherever the question can be answered by a rule. "Did the tests pass" and "is coverage above eighty percent" are rules. "Is the business ready for this" is not.

A gate that is always approved is theatre. If nobody has ever rejected it, it is adding delay and an audit record and nothing else. Either it encodes a real check, in which case automate it, or it should be removed.

Separation of duties is a real requirement in regulated work. The person who wrote the change is not the person who approves the release, and the pipeline records who did what. That is a legitimate reason for a human gate, and it is worth naming as the exception rather than pretending every gate is waste.

Control	Use it for
Required reviewers	Production, when someone must accept the risk
Wait timer	A soak period between canary and full rollout
Branch restriction	Only <code>main</code> reaches production
Change freeze	A known high-traffic window

Every gate is also a delay, and delay makes releases bigger. A bigger release is a riskier one, so a gate added to reduce risk can increase it. The honest question about any gate is what it has actually caught.

17. Deployment automation

Deployment automation is putting a built artifact into an environment with no manual steps.

What it replaces is a document: copy the jar to the server, stop the service, swap the file, start it, check the log. That works and it is not repeatable, because it depends on somebody following it exactly at a stressful moment.

Approach	How it works	Used by
Push	The pipeline connects to the target and installs	Ansible, a shell script
Pull	The target watches a repository and applies changes	Argo CD, Flux
Platform	The pipeline calls an API and the platform does it	<code>kubectl</code> , ECS, Cloud Run

Three properties make it trustworthy:

It is idempotent. Running it twice leaves the same result, so a retry after a network failure is safe.

Configuration comes from outside. The same artifact is promoted through every environment and told where it is at start, which is what makes a staging test mean something about production.

Rollback is the same mechanism. Redeploying the previous artifact, not building a fix under pressure. A separate rollback path is one more thing to maintain and one more thing to be wrong.

```
./deploy.sh --env=production --version=1.4.2
```

18. Observability after deployment

A green pipeline says the change passed every check that was written. Whether it works is a question only production answers.

Signal	Answers	Typical tool
Logs	What happened in one service	Loki, ELK, Cloud Logging
Metrics	How much, how fast, how often	Prometheus, Datadog
Traces	Where the time went across services	Jaeger, Tempo

Mark the deployment on the dashboards. A deployment marker is a line on the graph at the moment of release, and it turns "the error rate rose at 14:32" into "the error rate rose when we deployed". Without it, correlating a change with a symptom is guesswork.

Watch four numbers for the first ten minutes:

Number	Why
Error rate	The fastest signal that the release is wrong
Latency at p99	Averages hide the failure that matters
Saturation	Memory and CPU against their limits
Throughput	A drop can mean requests are not arriving at all

A health check is not monitoring. A readiness probe says the process answers; it says nothing about whether the answers are correct. A service can be perfectly healthy and returning errors to every user.

An SLO turns all of this into a decision. With a target for availability or latency, the error budget says whether there is room to keep releasing or whether the next change should be a fix. That is what stops the argument about whether a regression is serious enough to act on.

Automated rollback needs exactly this. Progressive delivery tools watch the canary's metrics against the baseline and undo the release on their own. Without observability that decision cannot be made by a person either, which is why this section is a prerequisite for Continuous Deployment rather than a nicety.

19. Rollback and failure recovery

A rollback is a deploy of the previously known-good artifact. It is not a special mode, and it should use the same path as any other deploy.

```
./deploy.sh --env=production --version=1.4.1
```

Roll back first, diagnose afterwards. Restoring service takes a minute; a forward fix takes a build, a test run and a deploy, all done under pressure by someone who does not yet know the cause.

Recovery	Use when	Time
Feature flag off	The change is behind a flag	Seconds, no deploy
Rollback	The previous version was fine	Minutes
Forward fix	Rolling back is impossible, usually because of data	Longer

A feature flag is the fastest rollback there is, because nothing is deployed. This is the strongest argument for putting risky changes behind one.

The database is what makes rollback hard. Application code reverts; a migration that dropped a column does not. The answer is to make schema changes backward compatible, in two releases:

```
Release 1  add the new column, write to both, read the old one
Release 2  read the new column
Release 3  stop writing the old one, then drop it
```

At every point the previous version of the application still works against the current schema, which is exactly what makes a rollback safe.

Limit who sees a bad release. A canary sends the new version a small share of traffic, so a failure affects a fraction of users and the decision is made on real data rather than on a staging guess.

Practice it. A rollback path that has never been used will not work the day it is needed: the old artifact was pruned, the script has drifted, or nobody knows the previous version number. Rolling back deliberately, on a normal day, is the only way to know.

The metric is mean time to recovery, not mean time between failures. Failures are inevitable; how long they last is a choice, and it is the number that a pipeline design is really optimizing.

20. How would you design a CI/CD pipeline?

This is the question the rest of the document is preparation for. It is open ended on purpose, and the answer is judged on structure rather than on tools.

Ask before designing. Four questions change the whole shape:

Question	Why it changes the design
What is being deployed?	A jar, a container and a mobile app are different pipelines
Where does it run?	Kubernetes, a VM and a managed platform differ at the last step
How often does the team release?	Once a quarter and forty times a day need different controls
Is there a compliance requirement?	Approvals and audit records stop being optional

Then state the principles, because these are what the design is derived from:

Everything in Git	the pipeline, the manifests, the configuration
Build once	one artifact, promoted through every environment
Fail fast	cheapest and most likely failure first
No manual steps	including rollback
Config at run time	one artifact, told where it is at start

Then walk the pipeline. Two lanes, because a pull request and a merge should not do the same work:

On pull request	build → unit tests → static analysis	~5 minutes
On merge to main	build → unit → analysis → package → scan → publish → deploy staging → e2e → [gate] → deploy production → watch	

A worked answer for a Java service:

Stage	Does	Gate
Build	<code>./mvnw -B verify</code> , cached <code>~/.m2</code>	Compile and unit tests pass
Analyze	Sonar, dependency scan, secret scan	Quality gate on new code
Package	Jar, then an image tagged with the commit sha	—
Scan image	Trivy against the pushed digest	No new critical findings
Publish	Push to the registry	—
Deploy staging	Same script as production	Deployment succeeds
Acceptance	End-to-end tests against staging	All pass
Approve	Environment protection rule	A person, if required
Deploy production	Canary, then full rollout	Metrics stay within the budget

Finish with the parts that are not the happy path, because this is where most answers stop early and where the follow-up questions come from:

Concern	The answer to give
Secrets	In the CI secret store or a vault, referenced by name
Rollback	Redeploy the previous digest; migrations are backward compatible
Observability	Deployment markers, error rate and p99 watched after release
Flaky tests	Quarantined, not retried
Speed	Dependency cache, parallel independent stages, a fast PR lane
Artifact retention	Keep everything production has run

Say what we would leave out. A three-person team shipping an internal service does not need a change advisory board, a canary or a separate performance environment. Naming the thing that is not worth its cost, and why, is what separates a design from a list of best practices.

A good closing sentence: the pipeline's job is to make the safe thing the easy thing, so that the fastest way to get a change to production is also the way that runs every check.

The pieces together

Piece	What it answers
Continuous Integration	Does the merged code still work?
Continuous Delivery	Could we release this right now?
Continuous Deployment	Did it release itself?
Pipeline	The stages, in order, stopping at the first failure
Fail fast	Why the cheap stages come first
Build once	Why the tested artifact is the deployed artifact
Build automation	One command, the same on every machine
Pipeline in Git	Why the build is reviewed like the code
Caching	Why a fresh runner is not slow
Test layers	Which suite should have caught this bug
Quality gate	The pass or fail rule, applied to new code
Dependency scanning	The risk in code we did not write
Image hardening	Minimal base, pinned digest, non-root, no secrets
Artifact	The one built thing that every environment installs
Snapshot or release	Whether it can change under us

Piece	What it answers
Gate	A rule, or a person, and which one it should be
Deployment automation	No manual steps, and rollback on the same path
Observability	Whether the release actually worked
Rollback	The previous artifact, on the same path, practised
<code>pom.xml</code>	What the project is, needs, and how it is built

For example, a change to the payroll service. The push triggers the pipeline, which runs `./mvnw -B verify` and finishes the unit tests in four minutes. Static analysis and the dependency scan run beside it. The jar is packaged once, published with the commit sha in its name, and deployed to staging, where the acceptance tests run against it. At that point the team has Continuous Delivery: the same artifact is sitting ready, and somebody decides when it goes further. Turning on the automatic last step would make it Continuous Deployment, and the work needed to be comfortable with that is feature flags, monitoring and a rollback that has been tested.

The question worth being able to answer is which of the three a team is actually doing, and what the next one would cost. Naming the missing piece, usually the tests or the rollback, is a better answer than describing the tools.